

Selenium - Automatización Web Santander

Automation Testing Introduction and Overview of Selenium

¿Qué es el concepto?

Bueno lo que hicimos fue automatizar que básicamente deja que un programa o un script pueda controlar nuestro navegador y así hacer pruebas en una página web que son automáticas y rápidas, para no estar dando clics manualmente cada vez que hay cambios en estas páginas. Usamos Selenium Web driver como nuestra herramienta principal que nos permitió conectar nuestro código en Java con el navegador (en esta práctica fue Chrome) para darle instrucciones reales como abrir la página, dar clics o validar textos.

En mi código

Se ve en las dependencias de mi archivo pom.xml (selenium-java, testng) y en mi clase BaseTest.java, donde creamos y arrancamos la sesión de ChromeDriver con `driver = DriverFactory.createChromeDriver();` y luego cerramos todo al terminar con `driver.quit();`

Comentarios personales:

Recuerdo hacer algunas de estas pruebas en universidad, siento que el testeado automatizado es una maravilla pues hacer los tests no lleva mucho tiempo y te salva de hacer pruebas repetitivas por horas, es muy transparente que al momento de que una prueba falla los reportes te hacen saber exactamente en qué línea fue el crasheo, y con lo que implementamos en esta práctica incluso te daba una captura de pantalla en la interfaz con el error.

Automation Fundamentals (Fundamentos de Automatización)

¿Qué es el concepto?

Son las reglas y la estructura base para que las pruebas funcionen de forma limpia y ordenada. Incluye preparar el terreno antes de cada prueba (precondiciones con `@BeforeMethod`), limpiar todo al terminar (postcondiciones con `@AfterMethod`), y verificar que la página realmente hizo lo que esperábamos mediante aserciones (`Assert.assertTrue`).

En mi código

- En BaseTest.java con las anotaciones `@BeforeMethod` (donde maximizo la pantalla y abro <https://www.santander.com.mx/>) y `@AfterMethod` (para cerrar el navegador).

- En mis clases de prueba (MenuPersonasTest, etc.), donde uso las anotaciones @Test de TestNG y validaciones de URL.

Comentarios personales:

Aquí di con mi primer error, si me tomó algo de tiempo pues mi primera prueba no pasaba, el famoso NullPointerException. Me pasó porque quise maximizar la ventana (driver.manage().window().maximize()) antes de haber creado el driver. Aprendí que en Java el orden de ejecución si importa: primero creas el objeto y luego le pides acciones. La razón por la que decidí maximizar es porque la página Santander es responsiva y en caso de moverla accidentalmente al ejecutar las pruebas su menú desplegable ya no coincidía con mi código de test.

3. Finding Elements (Localizadores / Locators)

¿Qué es el concepto?

Es la forma en que le decimos a Selenium cómo encontrar un botón, enlace o texto dentro de todo el código HTML de la página. Existen varios tipos: por id, por name, por clase, por selectores CSS o por XPath. El XPath relativo (//) es de los más fuertes porque nos deja buscar por texto visible (contains(., 'texto')), por atributos específicos (@href, @aria-label) o usando condiciones lógicas como el operador OR (|).

¿Dónde se ve en mi código?

En todas mis clases de pages/, algunos ejemplos:

- Por accesibilidad (aria-label): `private By tabPersonas = By.xpath("//button[@aria-label='personas']");`
- Por fragmento de URL (@href): `private By linkCreditoAutomotriz = By.xpath("//a[contains(@href, 'automotriz')]");`
- Por texto visible: `private By linkTarjetasCredito = By.xpath("//a[contains(., 'Tarjetas de crédito')]");`
- XPath Híbrido (con |): Lo usé en los botones principales para que encuentre el elemento ya sea que aparezca como botón o como texto: `//button[@aria-label='personas'] | //a[contains(text(), 'Personas')].`

Comentarios personales:

Santander fue un poco frustrante porque usa clases dinámicas y raras (tipo `sc-463fd39b-4`). Aprendí a la mala que jamás debo usar esas clases porque cambian solas pero tiene sentido al ser una página importante. También, cuando inspeccionaba con la consola de herramientas de desarrolladores, la página se encogía y cambiaba a modo móvil mostrándome botones tipo desplegable, pero al maximizar en pantalla completa el menú volvía a ser el de escritorio. La mejor solución fue apoyarme en los `@href` y los textos visibles.

How to Handle Elements (Manejo de Elementos y Esperas)

¿Qué es el concepto?

Es cómo interactuamos con lo que encontramos: dar clics (`.click()`), escribir (`.sendKeys()`) o seleccionar de listas desplegables. También nos abarca el manejo de tiempos, las esperas explícitas (`WebDriverWait`), que le dicen a Selenium que espere hasta que un elemento esté listo, y el uso de `JavascriptExecutor`, que sirve para forzar acciones cuando la interfaz gráfica tiene capas transparentes o animaciones que estorban.

¿Dónde se ve en mi código?

- En `BasePage.java` tengo centralizado mi `WebDriverWait` wait configurado a 12 segundos.
- Creé mi propio método `clickJS` usando `JavascriptExecutor`
- En `MenuPersonasPage` y las demás páginas, donde alterné entre el `click()` normal para abrir el menú y `clickJS()` para los sub-enlaces.

Comentarios personales:

Este fue mi mayor aprendizaje de la práctica. Me salieron errores de `TimeoutException` y `StaleElementReferenceException` porque el menú tiene animaciones y elementos ocultos en el DOM. Descubrí que la mejor estrategia fue intercalar entre ambos clics: usar el clic normal para los botones que abren el menú y `clickJS` para los enlaces finales, porque ese no se trababa con las animaciones.

Selenium Installation & Setup

- **¿Qué es el concepto?**

Es la preparación técnica inicial: tener configurado Java (JDK), agregar las dependencias correctas en el pom.xml, configurar el navegador con sus opciones y asegurarse de que el archivo binario del navegador (chromedriver) se comuniquen sin problemas con el código.

- **¿Dónde se ve en mi código?**

En mi clase DriverFactory.java y en el pom.xml. Además, en el método setUp() de BaseTest.java donde configuro la ventana con driver.manage().window().maximize() para que todas las pruebas corran siempre en resolución de escritorio.

- **Comentarios personales:**

En una parte de las pruebas se me quedó la pantalla en blanco con una URL que decía data:,. Me asusté pensando que se había roto el proyecto, pero descubrí que pasa cuando se quedan procesos fantasmas de ChromeDriver que se traban en el Administrador de Tareas, lo que nos explicó el profesor Carlos. Matando los procesos en la terminal volvió a levantar.

Selenium Page Object Model (POM) & Best Practices

- **¿Qué es el concepto?**

Es un patrón de diseño que dice que nunca debemos mezclar los localizadores y clics con las pruebas en un solo archivo. Se divide en dos partes:

- Pages: Clases que representan cada página o menú, donde se guardan los localizadores (By) privados y los métodos de acción públicos.
- Tests: Clases donde solo escribimos la historia de la prueba (dar clic, validar) usando los métodos que definimos en las Pages.

- **¿Dónde se ve en mi código?**

- Paquete pages: Creé MenuPersonasPage, MenuEmpresasPage, MenuPymesPage, MenuBancaPrivadaPage y MenuAcercaDelBancoPage, todas heredando de BasePage.
- Paquete tests: Creé MenuPersonasTest, MenuEmpresasTest, etc., todas heredando de BaseTest.

- Archivo testng.xml: Aquí agrupé las clases para poder ejecutar toda la suite una por una.

Comentarios personales:

Al principio se siente como crear muchos archivos para cosas simples, pero cuando tuve que corregir los errores de los clics, agradecí tenerlo así, incluso pensé en refactorizar Personas por categorías. Solo tuve que cambiar el método en MenuPersonasPage una sola vez y no tuve que reescribir toda mi prueba.